

Machine Learning

Módulo 1 — Introducción al Machine Learning
Apuntes de las 10 clases

Elaborado a partir de las transcripciones de clase

Contenido

Apuntes — ML Zoomcamp, Módulo 1: Introducción al Machine Learning	5
Índice	5
Hilo conductor del módulo	5
Conceptos que atraviesan todo el módulo	6
 1.1 — Introducción al Machine Learning	 7
El ejemplo: predecir el precio de un auto	7
De dónde sale el conocimiento	7
Vocabulario	7
El flujo completo	7
Precisión: el promedio, no el caso	8
Cómo se cierra el círculo en el producto	8
Definición resumida	8
 1.2 — Machine Learning vs sistemas basados en reglas	 9
El camino de las reglas	9
El camino del ML	9
La diferencia de fondo	10
 1.3 — Aprendizaje supervisado	 11
La idea	11
Notación formal	11
El objetivo	11
Los tipos de problema supervisado	11
Resumen	12
 1.4 — CRISP-DM: cómo se organiza un proyecto de ML	 13
1. Business understanding — entender el problema	13
2. Data understanding — qué datos tenemos	13
3. Data preparation — preparar los datos	14
4. Modeling — entrenar modelos	14
5. Evaluation — ¿alcanzamos el objetivo?	14

6. Deployment — poner el modelo en producción	14
Iterar	15
1.5 — El proceso de selección de modelos	16
El problema: evaluar como si fuera el futuro	16
Solución: esconder parte de los datos	16
Cómo se mide	16
El problema de las comparaciones múltiples	17
Solución: tres particiones	17
El proceso, paso a paso	18
Refinamiento: no desperdiciar el validation set	18
1.6 — Entorno de trabajo (GitHub Codespaces, edición 2024)	19
Crear el entorno	19
Instalar las librerías	19
Levantar Jupyter	19
Tip de la clase: acortar el prompt	19
Flujo de trabajo con git	20
Entregar la tarea	20
1.7 — Introducción a NumPy	21
Crear arrays	21
Acceder y modificar elementos	21
Arrays multidimensionales	21
Números aleatorios	22
Operaciones element-wise	22
Operaciones de resumen (summarizing)	23
1.8 — Repaso de álgebra lineal	24
Operaciones simples con vectores	24
1. Producto vector-vector (dot product)	24
2. Producto matriz-vector	25
3. Producto matriz-matriz	25
Matriz identidad	26
Matriz inversa	26

1.9 — Introducción a Pandas **27**

Crear un DataFrame	27
Lo primero al abrir un dataset	27
Acceder a columnas (Series)	27
El índice	28
Operaciones element-wise	29
Operaciones con strings	29
Operaciones de resumen	29
Valores faltantes	30
Group by	30
Conversiones	30

1.10 — Resumen de la sesión 1 **31**

1.1 — Introducción al ML	31
1.2 — Reglas vs machine learning	31
1.3 — Aprendizaje supervisado	31
1.4 — CRISP-DM	31
1.5 — Selección de modelos	31
1.6 — Entorno	31
1.7 — NumPy	31
1.8 — Álgebra lineal	32
1.9 — Pandas	32
Qué viene después	32

Apuntes — ML Zoomcamp, Módulo 1: Introducción al Machine Learning

Apuntes tomados de las transcripciones de las 10 clases del módulo 1.

Índice

#	Clase	Idea central
1.1	Introducción al ML	Features + target → modelo → predicciones
1.2	ML vs sistemas de reglas	Las reglas no escalan; el ML aprende los patrones solo
1.3	Aprendizaje supervisado	$g(X) \approx y$; regresión, clasificación, ranking
1.4	CRISP-DM	Las 6 fases de un proyecto de ML; iterar siempre
1.5	Selección de modelos	Split train/validation/test y el problema de comparaciones múltiples
1.6	Entorno (Codespaces)	Ambiente de trabajo y entrega de tareas
1.7	NumPy	Arrays, operaciones element-wise, agregaciones
1.8	Álgebra lineal	Productos vector-vector, matriz-vector, matriz-matriz, inversa
1.9	Pandas	DataFrames, Series, filtrado, group by, valores faltantes
1.10	Resumen de la sesión 1	Repaso de todo lo anterior

Hilo conductor del módulo

El módulo arma, de lo conceptual a lo práctico, la base para el módulo 2 (regresión lineal aplicada a predecir el precio de un auto):

1. **Qué es el ML** (1.1–1.3): extraer patrones de datos en vez de escribir reglas a mano.
2. **Cómo se organiza un proyecto** (1.4–1.5): CRISP-DM para el proceso completo, y el proceso de selección de modelos para la parte de modelado.
3. **Herramientas** (1.6–1.9): entorno, NumPy, álgebra lineal y Pandas.

Conceptos que atraviesan todo el módulo

- **Features (X)**: todo lo que sabemos del objeto. Matriz: filas = observaciones, columnas = características.
- **Target (y)**: lo que queremos predecir. Vector.
- **Modelo (g)**: la función que se entrena para que $g(x)$ se parezca lo más posible a y .
- **Entrenar (fit)**: el proceso de obtener g a partir de x e y .
- El modelo acierta **en promedio**, no caso por caso.

Nota: las transcripciones son automáticas y traen errores de reconocimiento («crisp them» por CRISP-DM, «psyt learn» por scikit-learn, «palm/sperm» por spam, etc.). En estos apuntes los términos van corregidos.

1.1 — Introducción al Machine Learning

El ejemplo: predecir el precio de un auto

Un sitio de avisos clasificados de autos. Cuando alguien publica su auto, llega al campo **precio** y se traba:

- Precio muy alto → nadie compra.
- Precio muy bajo → deja plata sobre la mesa.

La solución manual es mirar avisos similares y estimar. La pregunta del curso es: ¿cómo ayudamos al usuario a encontrar el precio justo? Con machine learning.

De dónde sale el conocimiento

Ya tenemos datos de autos publicados: precio, año de fabricación, marca, kilometraje, modelo, número de puertas, etc.

Un **experto** de una concesionaria puede estimar el precio mirando esas mismas características. ¿Cómo aprendió? Mirando muchísimos casos anteriores y extrayendo patrones: los autos viejos valen menos, más kilometraje baja el precio, una BMW cuesta más que una Volkswagen.

La idea clave: si un experto puede aprender esos patrones de los datos, un modelo también.

Vocabulario

Término	Definición	En el ejemplo
Features	Todo lo que sabemos del objeto	Año, marca, kilometraje, modelo, puertas
Target	Lo que queremos predecir	El precio
Modelo	Lo que encapsula los patrones aprendidos	—

El flujo completo

Entrenamiento:

Features + Target → [algoritmo de ML] → Modelo

Uso (predicción):

Features (sin target) → [Modelo] → Predicción

Para autos nuevos no sabemos el precio: le pasamos al modelo solo las features y nos devuelve el precio estimado.

Precisión: el promedio, no el caso

El modelo **no acierta siempre el precio exacto** de un auto específico. Puede predecir de más o de menos. Pero en promedio, para un auto de ese año, esa marca y ese kilometraje, la predicción es razonablemente correcta.

Esto no es un defecto a corregir: es la naturaleza del ML y hay que tenerlo presente al comunicar resultados.

Cómo se cierra el círculo en el producto

1. El usuario llena el formulario con los datos de su auto.
2. Extraemos las features de ese formulario.
3. El modelo devuelve un precio.
4. Ponemos ese precio en el campo como sugerencia.
5. El usuario lo ajusta si quiere (subirlo o bajarlo).

El usuario queda contento porque se ahorró toda la investigación manual.

Definición resumida

Machine learning es el proceso de extraer patrones de los datos.

Los datos son de dos tipos: **features** (información sobre el objeto) y **target** (lo que queremos predecir). La salida del proceso es un **modelo**, que después se usa con features nuevas para producir predicciones.

1.2 — Machine Learning vs sistemas basados en reglas

El ejemplo de esta clase es un **detector de spam** para un sistema de correo: queremos mandar los mensajes no deseados a la carpeta de spam.

El camino de las reglas

Analizamos los mensajes marcados como spam y encontramos patrones:

- Todo lo que viene de `promotions@online.com` es spam.
- Si el asunto contiene «tax review» y el dominio es `online.com`, es spam.

Codificamos esas reglas en Python: unos cuantos `if`, se despliega y funciona.

Por qué se rompe

1. Aparece otro tipo de spam (mensajes que piden un depósito de 10 dólares).
2. Agregamos la regla: si el cuerpo contiene «deposit» → spam.
3. Un usuario legítimo escribe sobre un depósito real y su mensaje cae en spam.
4. Hay que agregar más reglas para distinguir los dos casos.
5. Y otra vez, y otra vez.

El spam cambia constantemente, así que las reglas hay que mantenerlas al día para siempre. El código crece, se vuelve imposible de mantener y cada cambio rompe algo en otro lado. Ahí es cuando toca preguntarse si hay otra herramienta.

El camino del ML

Cuatro pasos: **obtener datos** → **extraer features** → **entrenar el modelo** → **usarlo**.

1. Datos

El botón «marcar como spam» ya genera las etiquetas: los usuarios nos dicen qué es spam y qué no. Eso es exactamente el target que necesitamos.

2. Features

Features **binarias** (solo valen 0 o 1), por ejemplo:

1. ¿El asunto tiene más de 10 caracteres?
2. ¿El cuerpo tiene más de 10 caracteres?
3. ¿El remitente es `promotions@online.com` ?
4. ¿El remitente es (*otro remitente sospechoso*)?
5. ¿El dominio del remitente es `test.com` ?
6. ¿La descripción contiene la palabra «deposit»?

Punto importante: estas features **salen de las reglas que ya teníamos**. Por eso conviene empezar con un sistema de reglas y después reusar esas reglas como features del modelo. No hay que saltar directo al ML.

Cada email se codifica como un vector: `[1, 1, 0, 0, 1, 1]` con target `1` (spam). Se repite para todos los emails y queda el dataset.

3. Entrenar

Features + target entran al algoritmo; el proceso de entrenar también se llama **fit**. Sale el modelo.

4. Usar el modelo

El modelo devuelve **probabilidades**, no etiquetas: 0.8, 0.6, 0.1, 0.01, 0.7, 0.4. Se interpretan como «qué tan seguro estoy de que esto es spam».

Después hay que tomar la **decisión**, con un umbral (típicamente 0.5):

Probabilidad	Decisión	Destino
0.8	spam	carpeta spam
0.6	spam	carpeta spam
0.1	no spam	bandeja de entrada
0.01	no spam	bandeja de entrada
0.7	spam	carpeta spam
0.4	no spam	bandeja de entrada

La probabilidad y la decisión son cosas distintas: el modelo da la primera, el umbral produce la segunda.

La diferencia de fondo

Esta es la comparación clave del módulo — **qué es entrada y qué es salida**:

	Software tradicional	Machine learning
Entrada	Datos + código (reglas)	Datos + resultado (etiquetas)
Salida	Resultado (spam / no spam)	Modelo

En software tradicional el resultado está *hard-codeado* en las reglas. En ML, el resultado es una **entrada** al algoritmo: le mostramos qué es spam y qué no, y la salida es un modelo que después produce resultados para casos que no vimos.

1.3 — Aprendizaje supervisado

La idea

En los dos ejemplos anteriores (precio de autos, spam) nosotros actuamos como **profesores** del modelo: le mostramos ejemplos junto con la respuesta correcta. De esos ejemplos el algoritmo extrae patrones y después **generaliza** a casos nuevos.

Eso es «supervisado»: existe una **etiqueta** (el target) que supervisa el aprendizaje.

Notación formal

Símbolo	Nombre	Qué es
X (mayúscula)	Matriz de features (<i>feature matrix</i>)	Array 2D: filas = observaciones, columnas = features
y (minúscula)	Vector de target	Array 1D con el valor a predecir de cada fila
g	El modelo	La función que entrenamos

Si hace rato que no ves álgebra: **y** es simplemente un arreglo de números (`[0, 1, 0, 1, ...]`) y **X** es un arreglo de arreglos, o sea una tabla.

El objetivo

$$g(X) \approx y$$

Entrenar un modelo es encontrar la función **g** que, aplicada a la matriz de features, produzca algo lo más parecido posible al target.

Ese proceso de encontrar **g** es el **entrenamiento**. Todo el curso gira alrededor de qué forma concreta puede tomar **g** : regresión logística, árboles de decisión, redes neuronales, etc.

Ejemplo: **g** recibe año, marca y kilometraje, y devuelve un precio lo más cercano posible al real. Si el precio real era 11.000 y el modelo dijo 15.000, no acertó exacto, pero puede ser suficientemente cerca para nuestro propósito.

Los tipos de problema supervisado

Se distinguen **por el tipo de target**.

Regresión

La salida es un **número**.

- Precio de un auto: de 0 a $+\infty$.
- Precio de una casa a partir de m^2 , habitaciones, distancia al centro y al metro.

En general el rango puede ser cualquiera, incluso de $-\infty$ a $+\infty$.

Clasificación

La salida es una **categoría**.

- **Multiclase:** clasificar imágenes en auto / gato / perro. Pueden ser 3, 10 o 1000 categorías.
- **Binaria:** solo dos categorías. Es el caso del spam, y es *el subtipo más usado en la práctica*. El modelo devuelve una probabilidad entre 0 y 1, y el target es un arreglo de ceros y unos.

Vas a toparte sí o sí con un problema de clasificación binaria cuando trabajes con ML.

Ranking

Ordenar ítems por un puntaje. No se cubre en profundidad en el curso, pero conviene saber que existe.

- **Sistemas de recomendación:** se le asigna a cada producto un score de 0 a 1 (qué tan probable es que a este usuario le guste), se ordena y se muestran los primeros N.
- **Buscadores:** Google puntúa los documentos por relevancia (0.9, 0.85...) y muestra los más relevantes primero.
- **Búsqueda en e-commerce:** alguien busca «iphone» y hay que mostrarle lo más relevante *para esa persona*.

Resumen

X (feature matrix) $\rightarrow$ g (modelo) $\rightarrow$ $\hat{y} \approx y$ (target)

Según el tipo de target:

```

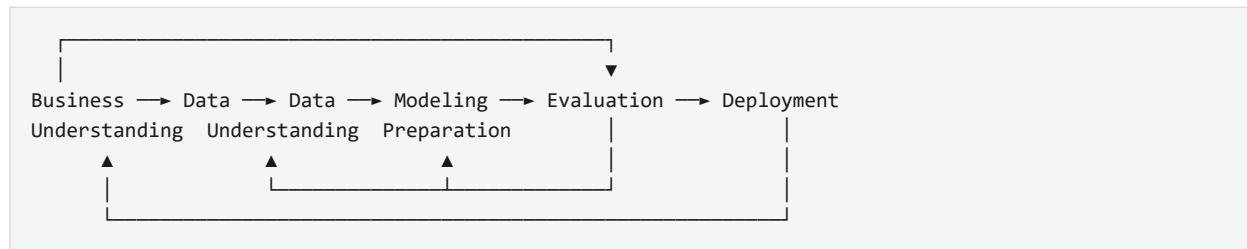
supervisado
├── regresión      → número
├── clasificación → categoría
│   ├── multiclase
│   └── binaria   ← la más usada
└── ranking       → orden por score
  
```

El curso se centra sobre todo en **clasificación**, con un capítulo de regresión.

1.4 — CRISP-DM: cómo se organiza un proyecto de ML

CRISP-DM = *C*Ross-*I*ndustry *S*tandard *P*rocess for *D*ata *M*ining. Metodología creada por IBM en los años 90. Es vieja, pero resistió el paso del tiempo: se sigue usando hoy casi sin modificaciones.

Son **6 fases**, y el punto central es que **se itera**: casi cualquier fase puede mandarte de vuelta a una anterior.



1. Business understanding — entender el problema

Objetivo: identificar el problema, entender si es importante y **definir cómo se mide el éxito**.

La pregunta obligatoria de esta fase:

¿De verdad necesitamos machine learning?

Muchos problemas no lo necesitan. «Si tenés un martillo, todo parece un clavo»: a veces basta con un sistema de reglas o una heurística, sin invertir tiempo y recursos en ML.

En el ejemplo del spam:

- ¿Cuántos usuarios se quejan? ¿Es un problema real o es un solo usuario? Esto también sirve para justificar la inversión de tiempo.
- ¿Es ML la herramienta correcta, o alcanza con reglas?
- Definir la métrica: no basta con «queremos reducir el spam». Hay que decir **cuánto**: «reducir los mensajes de spam en un 50 %». Sin un número no hay forma de decir después si el proyecto fue exitoso.

2. Data understanding — qué datos tenemos

Sin datos no hay machine learning. Hay que averiguar qué hay disponible y si alcanza.

Preguntas:

- **¿El dato es confiable?** El botón «marcar como spam», ¿funciona bien? ¿Se registra siempre el clic?
- **¿El dato es lo bastante bueno?** Si los usuarios marcan como spam mensajes que no lo son, el modelo va a aprender ese comportamiento y a reproducir el error. Hay que analizar los datos a mano.

- **¿Hay suficiente volumen?** Con 10 registros no se hace nada. Una salida legítima de esta fase es: «todavía no estamos listos, primero hay que juntar 2000 o 3000 registros».

Lo que se aprende acá puede cambiar el entendimiento del problema → **volver a la fase 1** y revisarla.

3. Data preparation — preparar los datos

Transformar los datos crudos en algo que se le pueda dar a un algoritmo.

- **Limpieza:** sacar ruido, corregir los casos mal etiquetados.
- **Pipelines:** una secuencia de pasos que toma datos crudos, aplica transformaciones y produce datos limpios. Es código que hay que escribir.
- **Formato tabular:** remitente, destinatario, asunto, cuerpo... y la columna con el target.
- **Extracción de features:** por ejemplo, la columna binaria «¿contiene la palabra deposit?».

El resultado es la **X** y la **y** de la clase anterior.

4. Modeling — entrenar modelos

Acá pasa el machine learning propiamente dicho.

El objetivo es **probar distintos modelos y elegir el mejor**.

Regresión logística, árboles de decisión, redes neuronales y muchos más — se ven a lo largo del curso. Cómo elegir el mejor es el tema de la clase 1.5.

Muy seguido acá se descubre que las features no alcanzan, o que hay problemas en los datos → **volver a la fase 3**.

5. Evaluation — ¿alcanzamos el objetivo?

Se vuelve a mirar la meta definida en la fase 1: dijimos «reducir el spam un 50 %». ¿Lo logramos? ¿Reducimos un 30 %? ¿Es suficiente?

Las salidas posibles incluyen una que suele olvidarse: **cancelar el proyecto**. Si la meta resultó inalcanzable, se hace una retrospectiva y se decide si vale la pena seguir invirtiendo tiempo o no.

Nota del instructor: hoy evaluación y despliegue van casi siempre juntos. En los 90 se evaluaba primero y se desplegaba después; ahora la forma de evaluar es desplegando.

Esto es la **evaluación online**: se prueba con usuarios reales, pero no con todos — por ejemplo con el 5 %. Si funciona bien, se extiende al resto.

6. Deployment — poner el modelo en producción

Si en la fase de modelado el foco era el machine learning, acá el foco es la **ingeniería**: que el servicio sea monitoreable, mantenible, escalable y confiable. Todas las buenas prácticas de ingeniería entran en juego en esta fase.

Iterar

No se termina en el despliegue. Se vuelve al *business understanding* con lo aprendido: ¿se puede mejorar? ¿Hace falta? Tal vez alcanza por ahora, y seis meses después aparecen problemas y arranca otra vuelta.

Consejo central de la clase: empezar simple. Hacer una iteración rápida y completa con algo sencillo, desplegar, aprender. Después volver y hacerlo un poco más complejo. Dos o tres iteraciones rápidas en vez de una larga: se pierde menos tiempo y se demuestra valor temprano.

1.5 — El proceso de selección de modelos

Esta clase profundiza la fase de **modeling** de CRISP-DM: cómo elegir el mejor modelo entre varios candidatos.

El problema: evaluar como si fuera el futuro

Cuando usamos un modelo en producción pasa esto:

- En **julio** entrenamos el modelo g con los datos que tenemos hasta ahí.
- En **agosto** lo aplicamos a mensajes que nunca vio.

Al evaluar queremos imitar esa situación: medir qué tan bien anda el modelo con datos que **no vio durante el entrenamiento**. No podemos viajar al futuro, pero sí podemos simularlo.

Solución: esconder parte de los datos

Tomamos el dataset completo, apartamos un ~20 % y hacemos de cuenta que no existe:

- **80 % → training**: lo usamos para entrenar (es «julio»).
- **20 % → validation**: no se usa para entrenar (es «agosto»).

```
X_train, y_train → entrenar → g
X_val           → g → ŷ (comparar con y_val)
```

Cómo se mide

El modelo devuelve probabilidades; con el umbral de 0.5 se convierten en predicciones y se comparan con el valor real:

$\hat{y}$ (prob.)	predicción	real	¿correcto?
0.8	1	1	si
0.7	1	0	no
0.6	1	1	si
0.1	0	0	si
0.9	1	1	si
0.6	1	0	no

4 de 6 correctas → **66 % de accuracy**.

Se repite con varios modelos:

Modelo	Accuracy
Regresión logística	66 %
Árbol de decisión	60 %
Random forest	67 %
Red neuronal	80 %

Parecería que ya está: elegimos la red neuronal. **Pero hay un problema.**

El problema de las comparaciones múltiples

Ejemplo ilustrativo: en vez de modelos, usemos **monedas**. El modelo es «tiro la moneda: cara = spam, cruz = no spam». Probamos con cinco monedas distintas sobre las mismas 5 observaciones de validación:

«Modelo»	Aciertos
Euro	20 %
Dólar	40 %
Zloty polaco	20 %
Rublo	20 %
Grivna ucraniana	100 %

La grivna «gana» con 100 % de accuracy. Pero sabemos que es puro azar: esa moneda simplemente produjo por casualidad la misma secuencia que el target.

Lo mismo puede pasar con modelos reales. Los métodos de ML son probabilísticos: uno de ellos puede tener suerte con ese subconjunto particular de datos, sin ninguna razón de fondo.

En estadística esto se llama **problema de comparaciones múltiples**: cuando se hace la misma comparación muchas veces contra el mismo conjunto de validación, alguno va a salir bien por azar. Si tomáramos otro 20 % de los datos, el resultado sería completamente distinto.

Solución: tres particiones

En vez de apartar un conjunto, apartamos **dos**:

train 60 %	val 20 %	test 20 %
------------	----------	-----------

El 60/20/20 no está escrito en piedra: pueden ser otras proporciones.

El **test set** se guarda y se olvida. Se hace toda la selección de modelos contra validation y, recién cuando ya elegimos el mejor, se lo aplica una sola vez al test set para confirmar que no tuvo suerte.

En el ejemplo: la red neuronal dio 80 % en validation y da 79 % en test. Son valores razonablemente cercanos → el modelo se comporta bien de verdad.

El proceso, paso a paso

1. **Dividir** el dataset en train / validation / test.
2. **Entrenar** el modelo con train.
3. **Validar** contra validation y registrar la métrica.
4. **Repetir** 2 y 3 para todos los modelos candidatos.
5. **Seleccionar** el mejor.
6. **Testear** el elegido contra el test set para confirmar.

Montar bien este proceso es una de las cosas más importantes del machine learning.

Refinamiento: no desperdiciar el validation set

Tal como está, el 20 % de validación termina «gastado»: solo sirvió para comparar. Se puede aprovechar mejor:

1. Hacer la selección de modelos normalmente.
2. Una vez elegido el mejor, **unir train + validation** (80 % de los datos).
3. **Reentrenar** el modelo con ese conjunto más grande → un **g** mejor, porque usa más datos.
4. Recién ahí evaluarlo contra el test set.

1.6 — Entorno de trabajo (GitHub Codespaces, edición 2024)

Clase práctica: cómo armar el ambiente de trabajo y entregar las tareas. La gracia de Codespaces es que **casi no requiere configuración**: es un entorno remoto que ya viene con casi todo.

El resumen de la sesión (1.10) menciona la alternativa clásica: instalar **Anaconda** localmente, que trae NumPy, Pandas, scikit-learn y Jupyter de una. También se puede levantar un servidor en AWS u otro cloud.

Crear el entorno

1. Crear un repositorio nuevo en GitHub (por ejemplo `ml-zoomcamp-homework`). Público, con README y `.gitignore` de Python.
2. Botón **Code** → pestaña **Codespaces** → *Create codespace on main*.
3. Se levanta un VS Code en el navegador. Se puede seguir ahí, o abrirlo en el VS Code de escritorio con *Open in VS Code Desktop*.

Para conectarse desde el VS Code de escritorio hace falta la **extensión GitHub Codespaces**. Normalmente la ofrece sola al abrir; si no, se busca en la pestaña de extensiones.

Por dentro es una máquina Linux (Ubuntu) remota, pero se trabaja como si fuera local: terminal con `Ctrl + ñ / `` `Ctrl + ``` (o *View* → *Terminal*), sistema de archivos normal, git funcionando.

Instalar las librerías

```
pip install jupyter numpy pandas scikit-learn seaborn
```

Con eso alcanza para los primeros cuatro o cinco módulos. Más adelante hacen falta `xgboost` y `tensorflow` , que se instalan igual.

Levantar Jupyter

```
jupyter notebook
```

Codespaces **detecta automáticamente** el puerto 8888 y lo redirige a la máquina local (pestaña **Ports**). Se abre el enlace y se pega el token —o la URL completa— que muestra la terminal.

Tip de la clase: acortar el prompt

El prompt de Codespaces es larguísimo y no deja ver lo que uno escribe:

```
PS1=> "
```

Flujo de trabajo con git

```
git status
git commit -am "readme update"
git push
```

Ejemplo del final de la clase: crear una carpeta `01-intro` , adentro un notebook, leer el CSV de la tarea con pandas...

```
import pandas as pd
df = pd.read_csv("<archivo de la tarea>.csv")
```

...completar, guardar, y después:

```
git add homework.ipynb
git commit -m "homework"
git push
```

Entregar la tarea

Se va al sitio del curso (*Machine Learning Zoomcamp* → *homework 1*), se completan las respuestas y se pega la URL del repositorio como enlace de la tarea.

1.7 — Introducción a NumPy

```
import numpy as np    # el alias np es la convención en data science
```

Crear arrays

```
np.zeros(10)          # array de 10 ceros
np.ones(10)           # array de 10 unos
np.full(10, 2.5)      # array de 10 elementos, todos 2.5

a = np.array([1, 2, 3, 5, 7, 12]) # desde una lista de Python
```

Rangos y secuencias

```
np.arange(10)         # [0, 1, ..., 9] - el límite superior es EXCLUSIVO
np.arange(3, 10)      # [3, 4, ..., 9]

np.linspace(0, 1, 11) # 11 números repartidos entre 0 y 1 (inclusive ambos)
```

`arange` es el equivalente del `range` de Python, pero devuelve un array de NumPy en vez de un iterador.

Truco de la clase: con `linspace(0, 1, 11)` los pasos quedan redondos (0.0, 0.1, 0.2...); con 10 no.

Acceder y modificar elementos

```
a[2]          # tercer elemento (los índices empiezan en 0)
a[2] = 10     # reemplazar
```

Arrays multidimensionales

```
np.zeros((5, 2))    # 5 filas, 2 columnas

n = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9],
])
```

El primer número de la tupla son **filas**, el segundo **columnas**.

```

n[0, 1]          # fila 0, columna 1 → 2
n[0, 1] = 20     # reemplazar un elemento

n[0]            # fila completa
n[2] = [1, 1, 1] # reemplazar una fila entera (la dimensión debe coincidir)

n[:, 1]         # columna 1 completa – los ":" significan "todas las filas"
n[:, 2] = [0, 1, 2] # reemplazar una columna

```

Para acceder a una columna **no se puede dejar el primer índice vacío**: hay que poner `:`.

Números aleatorios

```

np.random.rand(5, 2)      # uniforme estándar, entre 0 y 1
np.random.randn(5, 2)     # distribución normal estándar
np.random.randint(0, 100, size=(5, 2)) # enteros; el límite superior es exclusivo

```

Semilla (seed) — reproducibilidad

```

np.random.seed(2)
np.random.rand(5, 2) # siempre da lo mismo

```

Los números no son realmente aleatorios, son **pseudoaleatorios**: se generan a partir de una semilla. Fijándola, la secuencia es la misma en tu máquina y en la mía.

Puede variar levemente según la versión de NumPy o el sistema operativo, pero en general coincide.

Operaciones element-wise

La gran ventaja sobre las listas de Python: **no hace falta escribir un for**.

```

a = np.arange(5)      # [0, 1, 2, 3, 4]

a + 1                 # suma 1 a cada elemento
a * 2                 # multiplica cada elemento por 2
a / 100
a ** 2

b = (a / 2) * 10 + 10 # se pueden encadenar

```

Así es como se generan enteros entre 0 y 100: `np.random.rand(5, 2) * 100`.

Entre dos arrays

```

a + b    # suma elemento a elemento
a * b
a / b

```

Comparaciones (también element-wise)

```
a >= 2      # → array de booleanos [False, False, True, True, True]
a > b       # comparar dos arrays elemento a elemento
```

Filtrado con máscara booleana

```
a[a > b]    # devuelve solo los elementos donde la condición es True
```

Por dentro: `a > b` produce un array de booleanos, y usarlo como índice devuelve los elementos que cumplen la condición. Esto reaparece en Pandas.

Operaciones de resumen (summarizing)

A diferencia de las anteriores, devuelven **un solo número** en vez de un array:

```
a.min()     # mínimo
a.max()     # máximo
a.sum()     # suma
a.mean()    # promedio
a.std()     # desviación estándar
```

Funcionan igual sobre arrays bidimensionales (`n.sum()` , `n.min()` , ...).

La clase menciona que hay mucho más: por ejemplo obtener el mínimo de **cada fila** de un array 2D, u ordenar. Vale mirar la documentación.

1.8 — Repaso de álgebra lineal

Temas: operaciones con vectores, los tres tipos de multiplicación y la matriz inversa.

Nota de notación: en álgebra lineal los vectores se escriben como **columnas**. En NumPy se ven como filas. Es solo convención de escritura, no cambia nada en el código.

Operaciones simples con vectores

Multiplicar por un número: se multiplica cada elemento.

Sumar dos vectores: se suman elemento a elemento.

Ambas son exactamente lo que hace NumPy:

```
u + v      # suma element-wise
2 * v      # multiplicación por escalar
```

1. Producto vector-vector (dot product)

También llamado producto interno o escalar. **Ojo: no es la multiplicación element-wise de NumPy** — el resultado es **un número**, no un array.

Se multiplican los elementos uno a uno y después **se suman los resultados**:

$$u^T v = \sum_{i=1}^n u_i v_i$$

Ejemplo: $u = [2, 4, 5, 6]$, $v = [1, 0, 0, 2]$ → $2 \cdot 1 + 4 \cdot 0 + 5 \cdot 0 + 6 \cdot 2 = 2 + 12 = 14$

La notación $u^T v$

La **transpuesta** (T) convierte columnas en filas. Como para multiplicar necesitamos un vector fila por un vector columna, el producto punto se escribe $u^T v$. Es solo notación.

Implementación

```
def vector_vector_multiplication(u, v):
    assert u.shape[0] == v.shape[0]      # deben tener el mismo tamaño

    n = u.shape[0]
    result = 0.0
    for i in range(n):
        result = result + u[i] * v[i]
    return result
```

El **for** es la sumatoria de la fórmula; el cuerpo es lo que va adentro del símbolo Σ . La fórmula va de 1 a n , pero los arrays de NumPy se indexan de 0 a $n-1$.

En NumPy ya existe:

```
u.dot(v)    # → 14
```

2. Producto matriz-vector

Para cada **fila** de la matriz U se hace un producto punto con el vector v .

$$U \text{ (k filas} \times \text{n columnas)} \times v \text{ (n elementos)} = \text{vector de k elementos}$$

La dimensionalidad tiene que coincidir: el **número de columnas de U** debe ser igual al **número de elementos de v** .

```
def matrix_vector_multiplication(U, v):
    assert U.shape[1] == v.shape[0]

    num_rows = U.shape[0]
    result = np.zeros(num_rows)
    for i in range(num_rows):
        result[i] = vector_vector_multiplication(U[i], v)
    return result
```

En NumPy: `U.dot(v)`. NumPy se da cuenta solo de que, al invocarlo sobre un array 2D, tiene que hacer producto matriz-vector.

3. Producto matriz-matriz

Se descompone la matriz V en sus **columnas** y se hace un producto matriz-vector con cada una. Cada resultado es una columna del resultado final.

$$U \times V \rightarrow \text{columna } i \text{ del resultado} = U \cdot (\text{columna } i \text{ de } V)$$

Dimensiones del resultado: **filas de U $\times$ columnas de V** .

```
def matrix_matrix_multiplication(U, V):
    assert U.shape[1] == V.shape[0]

    num_rows = U.shape[0]
    num_cols = V.shape[1]
    result = np.zeros((num_rows, num_cols))

    for i in range(num_cols):
        vi = V[:, i]                # columna i de V
        Uvi = matrix_vector_multiplication(U, vi)
        result[:, i] = Uvi          # se asigna como columna
    return result
```

En NumPy: `U.dot(V)`.

La idea que unifica la clase: matriz-matriz se expresa como varias multiplicaciones matriz-vector, y matriz-vector se expresa como varias multiplicaciones vector-vector. Escritas en código, las fórmulas dejan de dar miedo.

Matriz identidad

Matriz **cuadrada** con unos en la diagonal y ceros en todo el resto.

```
I = np.eye(3)
```

Es el «número 1» de las matrices: $U \cdot I = U$ e $I \cdot U = U$, sin importar de qué lado se multiplique (con las dimensiones correspondientes).

Matriz inversa

La inversa de A , escrita A^{-1} , es la matriz tal que:

$$A^{-1} A = I$$

Solo las matrices cuadradas tienen inversa (mismo número de filas que de columnas).

```
Vs = V[[0, 1, 2]]          # tomar un bloque cuadrado
Vs_inv = np.linalg.inv(Vs)  # linalg = linear algebra

Vs_inv.dot(Vs)              # → matriz identidad
```

Por qué importa: la inversa es la pieza clave de la **regresión lineal**, que se ve en la sesión siguiente.

1.9 — Introducción a Pandas

Pandas es la librería de Python para manipular **datos tabulares**. Por debajo usa NumPy.

```
import pandas as pd
```

La abstracción principal es el **DataFrame** (una tabla). Cada columna de un DataFrame es una **Series**.

El dataset de la clase es un subconjunto del de precios de autos que se usa en la sesión 2: marca, modelo, año, motor, caballos de fuerza, cilindros, tipo de transmisión, estilo de vehículo y precio (MSRP).

Crear un DataFrame

Desde una lista de listas

```
data = [
    ["Nissan", "Stanza", 1991, 138.0, 4, "MANUAL", "sedan", 2000],
    # ...
]
columns = ["Make", "Model", "Year", "Engine HP", "Engine Cylinders",
           "Transmission Type", "Vehicle_Style", "MSRP"]

df = pd.DataFrame(data, columns=columns)
```

Sin el parámetro `columns`, Pandas nombra las columnas `0`, `1`, `2`, `3...` porque no sabe qué significan.

Desde una lista de diccionarios

```
data = [
    {"Make": "Nissan", "Model": "Stanza", "Year": 1991, ...},
    # ...
]
df = pd.DataFrame(data)
```

Acá no hace falta pasar `columns`: Pandas usa las **claves** de los diccionarios como nombres de columna.

Lo primero al abrir un dataset

```
df.head()      # primeras 5 filas
df.head(2)     # primeras 2
```

Es lo primero que hace el instructor después de cargar datos desde un CSV o una consulta SQL: mirar las primeras filas.

Acceder a columnas (Series)

```
df.Make          # notación de punto
df["Make"]       # notación de corchetes – equivalente
```

La notación de punto **no funciona** si el nombre tiene espacios o guiones ("Engine HP"). Para esos casos hay que usar corchetes.

Varias columnas a la vez

```
df[["Make", "Model", "MSRP"]] # doble corchete: una lista adentro de los []
```

Devuelve un DataFrame nuevo con solo esas columnas.

Agregar, reemplazar y borrar columnas

```
df["id"] = [1, 2, 3, 4, 5] # agregar (o reemplazar si ya existe)
del df["id"]               # borrar – igual que con un diccionario
```

El índice

Los números al costado de las filas son el **índice**.

```
df.index          # RangeIndex de 0 a 5 (el límite superior es exclusivo)
df.Make.index     # todas las Series comparten el índice del DataFrame
```

loc vs iloc

	Qué usa	Ejemplo
<code>df.loc[1]</code>	El índice por etiqueta	<code>df.loc[[1, 2]]</code>
<code>df.iloc[1]</code>	El índice posicional (0 a n-1)	<code>df.iloc[[1, 2]]</code>

Si se reemplaza el índice por otra cosa:

```
df.index = ["a", "b", "c", "d", "e"]

df.loc[1]          # ERROR: no existe ese índice
df.loc[["b", "c"]] # así sí
df.iloc[[1, 2]]    # el posicional sigue funcionando siempre
```

Volver al índice normal

```
df.reset_index()          # guarda el índice viejo en una columna "index"
df.reset_index(drop=True) # lo descarta

df = df.reset_index(drop=True) # hay que reasignar: NO modifica el DataFrame original
```

Patrón general de Pandas: casi todas las operaciones **devuelven un objeto nuevo** en vez de modificar el original. Si querés conservar el cambio, hay que reasignar.

Operaciones element-wise

Igual que en NumPy, pero sobre Series:

```
df["Engine HP"] / 100
df["Engine HP"] * 2
```

NaN indica un **valor faltante**; las operaciones lo dejan como está.

La diferencia con NumPy es que una Series tiene índice y nombre; por dentro es lo mismo.

Comparaciones y filtrado

```
df["Year"] >= 2015          # devuelve una Series booleana (True/False)

df[df["Year"] >= 2015]      # FILTRA: devuelve solo las filas donde es True
```

Son dos partes: la condición de adentro produce la máscara booleana, y los corchetes de afuera se quedan con las filas verdaderas.

```
df[df["Make"] == "Nissan"]

# combinar condiciones con & (and) y | (or)
df[(df["Make"] == "Nissan") & (df["Year"] >= 2015)]
```

Operaciones con strings

Algo que NumPy no tiene: NumPy es sobre todo para números, en Pandas hay strings todo el tiempo.

El accesor `.str` aplica un método de string a **todos** los elementos de la Series:

```
df["Vehicle_Style"].str.lower()          # a minúsculas
df["Vehicle_Style"].str.replace(" ", "_") # espacios → guiones bajos

# encadenables
df["Vehicle_Style"].str.replace(" ", "_").str.lower()

# recordar reasignar para que el cambio quede
df["Vehicle_Style"] = df["Vehicle_Style"].str.replace(" ", "_").str.lower()
```

Es un paso de preprocesamiento típico: normalizar mayúsculas e inconsistencias de espaciado.

Operaciones de resumen

```
df["MSRP"].mean()      # también min(), max(), sum(), std()

df["MSRP"].describe()  # count, mean, std, min, percentiles (25/50/75), max
df.describe()          # lo mismo para TODAS las columnas numéricas
df.describe().round(2) # más compacto de leer
```

`describe()` sobre el DataFrame ignora automáticamente las columnas de texto: no tiene sentido el promedio de «Make».

Variables categóricas

```
df["Make"].nunique()  # cuántos valores únicos hay
df.nunique()          # valores únicos por cada columna
```

Útil para ver de un vistazo, por ejemplo, que solo hay dos tipos de transmisión.

Valores faltantes

Importan mucho: para machine learning **no queremos NaN**.

```
df.isnull()          # DataFrame de booleanos: True donde falta el valor
df.isnull().sum()    # ← lo realmente útil: cuántos faltantes por columna
```

La suma se aplica por columna, así que da la cuenta de faltantes de cada una.

Group by

El equivalente de este SQL:

```
SELECT transmission_type, AVG(MSRP)
FROM cars
GROUP BY transmission_type;
```

es:

```
df.groupby("Transmission Type")["MSRP"].mean()
df.groupby("Transmission Type")["MSRP"].min()
df.groupby("Transmission Type")["MSRP"].max()
```

Primero se agrupa por una columna, después se calcula el agregado dentro de cada grupo.

Conversiones

```
df["MSRP"].values      # el array de NumPy que está por debajo
df.to_dict(orient="records") # de vuelta a lista de diccionarios
```

`to_dict(orient="records")` es útil para guardar a un archivo o pasarle los datos a otro sistema.

1.10 — Resumen de la sesión 1

Repaso rápido de todo el módulo.

1.1 — Introducción al ML

Ejemplo del precio de autos. **Features** = todas las características que conocemos del auto; **target** = lo que queremos predecir (el precio). Features y target entran a un algoritmo de ML y sale un **modelo**, que después se usa con autos cuyo precio no conocemos.

1.2 — Reglas vs machine learning

En un sistema de reglas los humanos encuentran los patrones a mano y los traducen a código. Se vuelve inmanejable con el tiempo. Con ML los modelos extraen los patrones solos: miran los datos de entrenamiento y usan estadística y matemática para descubrir qué sirve para decidir.

1.3 — Aprendizaje supervisado

Ambos ejemplos son aprendizaje supervisado porque tenemos el **target** (y): la información que queremos predecir y que ya conocemos. El modelo g extrae patrones de la matriz de features x y produce algo lo más cercano posible a y .

Tipos según el target: regresión (número), clasificación (categoría; multiclase o binaria) y ranking.

1.4 — CRISP-DM

El modelado ($g(x) = y$) es **solo una parte** del proceso completo. Alrededor están el entendimiento del negocio, el de los datos, la preparación (la x hay que armarla), y el despliegue — porque sin despliegue **ni el mejor modelo sirve de nada**.

1.5 — Selección de modelos

Se parte el dataset en **tres**: se usa validation para encontrar el mejor modelo y test para asegurarse de no haber elegido uno que quedó bien **por pura casualidad**.

1.6 — Entorno

No es tanto una clase como la preparación del ambiente: NumPy, Pandas, scikit-learn, Jupyter. La opción más fácil es Anaconda; también se puede usar GitHub Codespaces o un servidor en AWS u otro cloud.

1.7 — NumPy

Librería de Python para manipular datos numéricos (arrays), con las operaciones útiles para data science y machine learning.

1.8 — Álgebra lineal

Los tres productos: **vector-vector**, **matriz-vector** (u mayúscula por v minúscula) y **matriz-matriz**.

La relación entre ellos es lo importante: el producto matriz-matriz se expresa como un conjunto de productos matriz-vector, y el matriz-vector como un conjunto de productos vector-vector. Implementados en código, las fórmulas dejan de asustar.

1.9 — Pandas

Librería para procesar datos tabulares. La abstracción principal es el **DataFrame**.

Qué viene después

Esta sesión fue **abstracta**: qué se puede hacer con machine learning, con NumPy, con Pandas.

La siguiente es **práctica**: un proyecto completo prediciendo el precio de un auto (regresión lineal).